Tutorial 05: Diffuse, Ambient, Directional Lighting - Teaching Guide

Overview

This tutorial introduces **physics-based lighting** in GLSL. Instead of storing colour in the VBO, we now compute colour entirely from lighting equations inside the vertex shader. We implement the ambient and diffuse components of the classic **Phong lighting model** using a directional light (like the Sun) and surface normals.

Prerequisites and Setup

```
.\shader_tutorial_venv\Scripts\Activate.ps1
cd tutorials
python 05_DiffuseAmbientLighting.py
```

Expected Output: Three panels of a "bow window" lit by a white directional light. Each panel faces a slightly different direction, so you clearly see brightness differences driven by the angle between the surface normal and the light direction. A subtle red global ambient tint is always visible.

Interactive Controls:

- **ESC** - Exit

Learning Objectives

By the end of this tutorial, students will understand:

1. What ambient and diffuse lighting are and how they differ physically
2. How normals work as vertex attributes
3. The `gl_NormalMatrix` and why we need it
4. Lambert's Law and the dot-product calculation
5. How to write and call a GLSL helper function
6. How to organise many uniforms efficiently in Python
7. Eye-space vs model-space for lighting calculations

What's New in This Tutorial

Compared to Tutorial 04:

- ☑ **Normals** as vertex attributes
- ☑ `gl_NormalMatrix` for correct normal transformation

- ☑ **Ambient lighting** (global + per-light)
- ☑ **Diffuse lighting** (Lambertian dot product)
- ☑ **Custom GLSL helper function** (`phong_weightCalc`)
- ☑ **Vertex colour computed from physics** — no colour in VBO anymore
- ☑ **Iterative uniform/attribute setup** — loop instead of manual calls

---

What Is Lighting?

Real lighting is extremely complex. Every photon bounces off dozens of surfaces. A real-time GPU cannot simulate this exactly, so we use **approximations**.

The classic approximation used in games and visualisation for decades is the **Phong lighting model**, which splits light into three independent components:

Component	What it simulates	This tutorial?
Ambient	Light that has bounced so many times it comes from everywhere	☑ Yes
Diffuse	Direct light scattered equally in all directions (matte surfaces)	☑ Yes
Specular	Direct light reflected in a specific direction (shiny surfaces)	❌ Tutorial 06

Data Flow Comparison

Tutorial 04 (Tweening):

```
VBO: position + tweened + color → vertex shader → fragment shader
```

Tutorial 05 (Lighting):

```
VBO: position + normal → vertex shader (CALCULATE color from lighting) →
fragment shader
                              ↑
            6 uniforms: Global_ambient, Light_ambient, Light_diffuse,
                        Light_location, Material_ambient, Material_diffuse
```

The VBO no longer stores colour — colour is **computed on the GPU** from lighting equations.

Code Breakdown

1. Ambient Lighting

```
// Global: constant light always in the scene (even with no lights)
Global_ambient * Material_ambient
```

```
// Per-light: how much this light raises the ambient level
Light_ambient * Material_ambient
```

What is ambient light physically?

- Light that has scattered so many times in the environment it has no direction
- Used to avoid pitch-black shadows (which look unrealistic)
- Think: overcast sky, or a room with many light sources

The formula:

```
Ambient = (Global_ambient + Light_ambient) * Material_ambient
```

No geometry information is needed — position and normal don't matter. Every point on the surface gets the same ambient value.

Our values:

```
glUniform4f(Global_ambient_loc, 0.3, 0.05, 0.05, 1.0)  # Reddish global ambient
glUniform4f(Light_ambient_loc,  0.2, 0.2, 0.2,  1.0)   # Neutral per-light
ambient
glUniform4f(Material_ambient_loc, 0.2, 0.2, 0.2, 1.0)  # Reflects 20% of
ambient
```

Effect: Every surface shows a constant reddish-grey tint regardless of orientation.

---

2. Diffuse Lighting — Lambert's Law

```
float diffuse_weight = phong_weightCalc(
    normalize(EC_Light_location),
    normalize(gl_NormalMatrix * Vertex_normal)
);
```

What is diffuse light physically?

- Light scattered equally in ALL directions from a rough surface
- Snow, chalk, rough wood = very diffuse (matte)
- The brightness depends on the ANGLE between light and surface normal

Lambert's Law:

```
Diffuse_weight = max(0, dot(normal, light_direction))
               = max(0, cos(angle_between_them))
```

Visual:

```
Light direction → → → → →

Surface A (facing light):        Surface B (45° angle):      Surface C (away
from light):
      ↑ normal                         ↗ normal                    → normal
   ═══════                          ═══════                     ═══════
dot(N, L) = 1.0                  dot(N, L) = 0.71              dot(N, L) =
0.0
(full brightness)                 (71% brightness)              (no diffuse)
```

**Why max(0, ...)? ** — A surface facing away from the light would give a negative dot product. We clamp to 0 (no light) instead of a negative contribution.

---

3. The `phong_weightCalc` Helper Function

```glsl
float phong_weightCalc(
    in vec3 light_pos,     // normalised light direction
    in vec3 frag_normal    // normalised surface normal
) {
    float n_dot_pos = max( 0.0, dot( frag_normal, light_pos ) );
    return n_dot_pos;
}
```

GLSL function syntax compared to Python:

Feature	GLSL	Python
Return type	Declared before name: `float phong_weightCalc(...)`	Always `def`, return type inferred
Parameter direction	`in` (read-only), `out` (write-only), `inout` (both)	All parameters are references
Call	`phong_weightCalc(a, b)`	Same
Scope	Must be defined before use (no forward declarations without prototype)	Anywhere

Why a helper function?

- Reusable across multiple lights
- Tutorial 06 will add specular and reuse the same function
- Keeps `main()` readable

---

4. Normals as Vertex Attributes

```
attribute vec3 Vertex_normal;
```

What is a Normal?

- A vector perpendicular to the surface at a given point
- Length = 1.0 (unit vector) for lighting calculations
- Tells the shader "which direction is this surface facing?"

VBO layout for Tutorial 05:

```
Each vertex row:  [pos.x, pos.y, pos.z, norm.x, norm.y, norm.z]
Stride:           6 floats × 4 bytes = 24 bytes
Normal offset:    3 floats × 4 bytes = 12 bytes
```

Diagram:

```
Byte: 0          12         24 (next vertex)
      ├──────────┼──────────┤
      | pos xyz  | norm xyz |
      └──────────┴──────────┘
```

Per-vertex normals for smooth shading:
In the bow-window geometry, the left panel vertices all share the same normal (-1,0,1), the right panel shares (1,0,1). This gives each panel a uniform colour. For smooth curved surfaces (Tutorial 06), normals are averaged between adjacent faces so the surface looks continuous.

---

5. `gl_NormalMatrix` — Why Normals Need Special Treatment

```
vec3 EC_Light_location = gl_NormalMatrix * Light_location;
normalize(gl_NormalMatrix * Vertex_normal)
```

The problem:
Normals cannot be transformed by the same matrix as positions! If you scale an object, the normals also get scaled — but they should remain unit length and perpendicular to the surface.

The fix: `gl_NormalMatrix`

- Automatically derived from the model-view matrix
- Correctly handles rotation, scaling, and shearing
- Always produces a normal that is perpendicular to the transformed surface
- Type: `mat3` (3×3, because normals are directions, not positions)

Visual:

```
Original:           After uniform scale 2×:       After non-uniform scale:
  ↑ normal              ↑ normal (still OK          ↗ wrong!    ↑ correct
(NormalMatrix)
  |                     after NormalMatrix)
═════                 ═══════════
```

Rule: Always use `gl_NormalMatrix * normal`, never `gl_ModelViewMatrix * vec4(normal, 0.0)`.

---

6. Eye-Space Lighting

```
vec3 EC_Light_location = gl_NormalMatrix * Light_location;
```

We transform the light direction into **eye space** (camera-relative coordinates) before the dot product.

Why eye-space?

- Simplifies more complex lighting (specular needs to know where the camera is)
- In eye-space, the camera is always at `(0,0,0)` — constant, easy to use
- Most OpenGL lighting documentation assumes eye-space

Could we skip this? (Model-space calculation):

```
// Simpler — no NormalMatrix needed for uniform scale
float weight = phong_weightCalc(
    normalize(Light_location),
    normalize(Vertex_normal)
);
```

This works for simple cases but breaks when the model has non-uniform scaling.

---

7. The Full Lighting Equation

```
baseColor = clamp(
    (Global_ambient  * Material_ambient)
```

```
    + (Light_ambient   * Material_ambient)
    + (Light_diffuse   * Material_diffuse * diffuse_weight),
      0.0, 1.0
  );
```

Breaking it down:

Term	Description	Always on?
`Global_ambient * Material_ambient`	Scene-wide ambient baseline	Yes
`Light_ambient * Material_ambient`	Light's ambient contribution	Yes (if light enabled)
`Light_diffuse * Material_diffuse * diffuse_weight`	Directional diffuse	Only when surface faces light

Example calculation (centre panel, facing directly at light):

```
diffuse_weight ≈ 0.9  (nearly facing the light)

Global term: (0.3, 0.05, 0.05) * (0.2, 0.2, 0.2) = (0.06, 0.01, 0.01)  ← reddish
Light amb:   (0.2, 0.2, 0.2)   * (0.2, 0.2, 0.2) = (0.04, 0.04, 0.04)
Diffuse:     (1.0, 1.0, 1.0)   * (1.0, 1.0, 1.0) * 0.9 = (0.9, 0.9, 0.9) ← bright

Total:       (1.0, 0.95, 0.95) after clamp → near-white with slight red tint
```

Left panel (angled away from light):

```
diffuse_weight ≈ 0.3  (angled 72° from light)
Diffuse: (1.0, 1.0, 1.0) * 0.3 = (0.3, 0.3, 0.3) ← darker
Total: ambient + (0.3, 0.3, 0.3) → visibly darker than centre panel
```

---

8. Iterative Uniform/Attribute Setup

```python
for uniform_name in ('Global_ambient', 'Light_ambient', ...):
    loc = glGetUniformLocation(self.shader, uniform_name)
    setattr(self, uniform_name + '_loc', loc)

for attr_name in ('Vertex_position', 'Vertex_normal'):
    loc = glGetAttribLocation(self.shader, attr_name)
    setattr(self, attr_name + '_loc', loc)
```

Instead of writing `self.Global_ambient_loc = glGetUniformLocation(...)` six separate times, we loop over the names.

`setattr(obj, name, value)` is equivalent to `obj.name = value` but with the name as a variable.

After the loop:

- `self.Global_ambient_loc` → integer location for `Global_ambient` uniform
- `self.Light_ambient_loc` → integer location for `Light_ambient` uniform
- etc.

This pattern is common in real OpenGL code when dealing with many uniforms/attributes.

Complete Data Flow — One Vertex Through the Pipeline

Input (vertex 7 from VBO — centre-right panel):

```
Vertex_position = (1, 0, 1)
Vertex_normal   = (1, 0, 2)   ← un-normalised, will be normalised in shader
```

Vertex Shader:

```
1. gl_Position = MVP * (1, 0, 1, 1)  → clip-space position
2. EC_Light_location = NormalMatrix * (2, 2, 10) → eye-space light dir
3. normalised normal = normalize(NormalMatrix * (1,0,2)) ≈ (0.45, 0, 0.89)
4. diffuse_weight = max(0, dot(light_dir, normal)) ≈ 0.7
5. baseColor = clamp(
       (0.3,0.05,0.05)*(0.2,0.2,0.2)          ← global ambient
     + (0.2,0.2,0.2)*(0.2,0.2,0.2)            ← light ambient
     + (1,1,1)*(1,1,1)*0.7,                   ← diffuse
     0, 1)
   ≈ (0.76, 0.74, 0.74)  (bright grey with faint red tint)
```

Rasterisation: Interpolates `baseColor` across triangle.

Fragment Shader:

```
gl_FragColor = baseColor → pixel written to screen
```

Visual Explanation of the Bow Window

```
Top view (looking down Y axis):

              Light direction → →
                                  ↑ z-axis
Left panel    Centre panel    Right panel
\             |              /
 \  normal    |  normal     / normal
  ↖(-1,0,1) ↑(0,0,1)    ↗(1,0,1)
   \          |          /
    \─────────┼─────────/
   x=-1      x=0,z=1   x=2

Brightness:  ▒▒          ████        ▒▒
             dim          bright      medium
             (angled      (facing     (angled
              away)        light)      toward)
```

Why "bow window"?

- A bay/bow window has three panels at different angles
- Each panel faces a different direction
- Perfect for demonstrating how diffuse lighting depends on angle

---

Phong Lighting Model — Summary

```
Final Colour = Ambient + Diffuse + Specular

Ambient  = (Global_ambient + Light_ambient) * Material_ambient
Diffuse  = Light_diffuse * Material_diffuse * max(0, dot(N, L))
Specular = Light_specular * Material_specular * pow(dot(R, V), shininess)
                                                        ↑
                                                  Tutorial 06!
```

Where:

- `N` = surface normal (normalised)
- `L` = light direction (normalised)
- `R` = reflection of L around N
- `V` = view direction (camera direction)

Built-in GLSL Features Used

`gl_NormalMatrix`

- Type: `mat3`

- Value: `transpose(inverse(mat3(gl_ModelViewMatrix)))`
- Use: transform normals from model-space to eye-space

`dot(a, b)`

- Returns the dot product: `a.x*b.x + a.y*b.y + a.z*b.z`
- For unit vectors: equals `cos(angle_between_them)`
- Range: -1 (opposite) to +1 (same direction), 0 = perpendicular

`normalize(v)`

- Returns unit vector: `v / length(v)`
- Essential before dot products for lighting

`clamp(x, min, max)`

- Constrains x to [min, max]
- Used to keep final colour in [0, 1] range

`max(a, b)`

- Returns larger of a and b
- Used in `max(0.0, dot(...))` to prevent negative lighting

Common Student Questions

Q: Why does the global ambient have a red tint?
A: `glUniform4f(Global_ambient_loc, 0.3, 0.05, 0.05, 1.0)` — intentionally reddish so students can clearly see its contribution. A real scene might use a subtle grey or sky-blue.

Q: What if I don't want ambient light?
A: Set both `Global_ambient` and `Light_ambient` to `(0, 0, 0, 0)`. The dark side of objects will be completely black.

Q: Why pass un-normalised normals in the VBO?
A: The shader normalises them with `normalize(gl_NormalMatrix * Vertex_normal)`. Storing un-normalised normals in the VBO can be intentional (e.g., storing direction hints for smooth shading blends).

Q: Can I have multiple lights?
A: Yes! Call `phong_weightCalc` once per light and add the results together:

```
float w1 = phong_weightCalc(normalize(Light1_EC), normalize(normal));
float w2 = phong_weightCalc(normalize(Light2_EC), normalize(normal));
baseColor = ambient + Light1_diffuse * w1 + Light2_diffuse * w2;
```

Q: What's the difference between a directional and positional light?
A:

- **Directional (this tutorial):** Infinitely far away. All rays are parallel. No attenuation with distance. Good for the Sun.
- **Positional:** Has a 3D position. Rays diverge from that position. Brightness decreases with distance. Good for lamps.

Q: Why doesn't the fragment shader do anything?
A: In this tutorial, all lighting is done per-vertex. The varying `baseColor` is computed in the vertex shader and interpolated across the triangle. The fragment shader just outputs the already-computed colour. Tutorial 06 moves the calculation to the fragment shader.

Experiments to Try

1. Move the Light

```
glUniform3f(self.Light_location_loc, -2.0, 2.0, 10.0)  # light from the left
```

Effect: Different panels will now appear bright/dark.

2. Coloured Light

```
glUniform4f(self.Light_diffuse_loc, 1.0, 0.5, 0.0, 1.0)  # orange light
```

Effect: Diffuse component takes on the light's colour.

3. Remove Global Ambient

```
glUniform4f(self.Global_ambient_loc, 0.0, 0.0, 0.0, 0.0)
```

Effect: Back sides go darker; the red tint disappears.

4. Change Material Diffuse Colour

```
glUniform4f(self.Material_diffuse_loc, 0.2, 0.6, 1.0, 1.0)  # blue material
```

Effect: The panels appear blue, brightened by the light.

5. Visualise the Normal

```
// In vertex shader, replace the baseColor calculation:
vec3 n = normalize(gl_NormalMatrix * Vertex_normal);
```

```
baseColor = vec4(n * 0.5 + 0.5, 1.0);  // remap -1..1 to 0..1
```

Effect: Normals displayed as colours — great for debugging.

Debugging Tips

Problem: Everything is one flat colour

- Check `Vertex_normal` location is not -1
- Verify the VBO normal offset is 12 (not 0)
- Make sure the normal data isn't all zeros

Problem: Unexpected dark surface

- The panel may face away from the light — try moving the light
- Check the normal direction in the VBO data

Problem: All black except ambient

- `Light_diffuse` may be zero
- `diffuse_weight` may be zero — the normal and light direction are perpendicular

Visualise diffuse weight:

```
float w = phong_weightCalc(normalize(EC_Light_location),
normalize(gl_NormalMatrix * Vertex_normal));
baseColor = vec4(w, w, w, 1.0);  // grey-scale weight
```

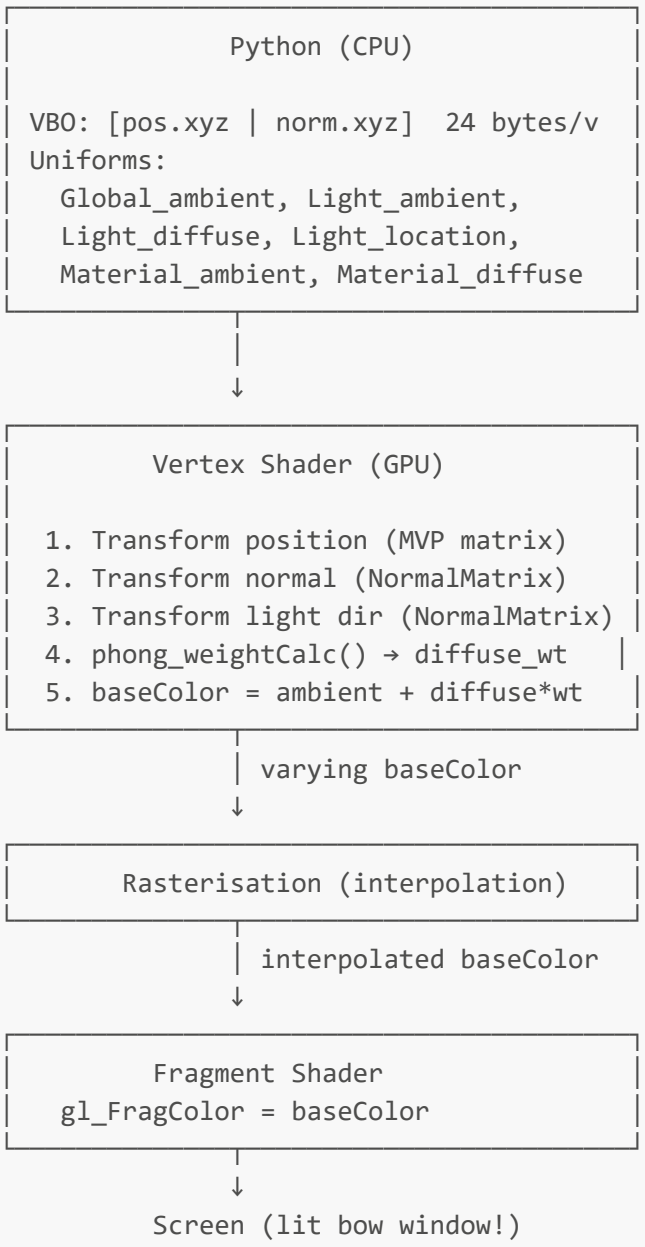
Performance Considerations

Vertex shader vs Fragment shader for lighting:

	Vertex Lighting (this tutorial)	**Fragment Lighting (Tutorial 06)**
Runs N times	Once per vertex	Once per pixel
Speed	Fast	Slower
Quality	Interpolated — can miss specular highlights between vertices	Per-pixel — accurate specular
Use case	Ambient + diffuse (smooth gradients)	Specular highlights, fine details

For diffuse-only lighting, vertex shading is usually good enough and much faster.

Summary Diagram



Next Steps

You now know:

- ☑ Tutorial 01: Basic shaders and VBOs
- ☑ Tutorial 02: Varying values and colour interpolation
- ☑ Tutorial 03: Uniform values and fog calculations
- ☑ Tutorial 04: Custom attributes and tween animation
- ☑ Tutorial 05: Ambient + diffuse lighting, normals, NormalMatrix

Coming in Tutorial 06:

- Specular highlights (Blinn-Phong half-vector method)
- Moving lighting to the **fragment shader** for smooth per-pixel results
- **Indexed geometry** — efficient rendering of shared vertices
- Procedurally generated **sphere** geometry